

jucheng1992 / ctmle

<> CodeIssues 1Pull requestsAgentsActionsProjectsSecurity and qualityInsights

On April 24 we'll start using GitHub Copilot interaction data for AI model training unless you opt out. [Review this update](#) and manage your preferences in your [GitHub account settings](#).

Watch 2

Collaborative Targeted Maximum Likelihood Estimation

5 stars

3 forks

2 watching

Branches

Activity

Tags

Public repository

master

1 Branch

0 Tags

Go to file

t

Go to file

Add file

Code

jucheng1992	update reference	18de559 · 7 years ago	
.Rproj.user/D08D4C24	fix bugs	7 years ago	
R	fix bugs	7 years ago	
man	fix bugs	7 years ago	
tests	update reference	7 years ago	
vignettes	fix indent	7 years ago	
.DS_Store	update reference	7 years ago	
.Rbuildignore	update reference	7 years ago	
.Rhistory	fix bugs	7 years ago	
.gitignore	md	9 years ago	
CRAN-RELEASE	update reference	7 years ago	
DESCRIPTION	update reference	7 years ago	
NAMESPACE	more document	9 years ago	
NEWS.md	submit to CRAN	9 years ago	
README.Rmd	update reference	7 years ago	
README.md	small changes	9 years ago	
cran-comments.md	update reference	7 years ago	
ctmle.Rproj	first committe	9 years ago	

README

CRAN 0.1.2

downloads 293/month

downloads 33K

Collaborative Targeted Maximum Likelihood Estimation

Collaborative Targeted Maximum Likelihood Estimation (C-TMLE) is an extension of Targeted Maximum Likelihood Estimation (TMLE). It applies variable/model selection for nuisance parameter (e.g. the propensity score) estimation in a 'collaborative' way, by directly optimizing the empirical metric on the causal estimator.

In this package, we implemented the general template of C-TMLE, for the estimation of the average treatment effect (ATE).

The package also offers convenient functions for discrete C-TMLE for variable selection, and LASSO-C-TMLE for model selection of LASSO, in estimation of the propensity score (PS).

Installation

To install the CRAN release version of `ctmle` :

```
install.packages('ctmle')
```



To install the development version (requires the devtools package):

```
devtools::install_github('jucheng1992/ctmle')
```



C-TMLE for variable selection

In this section, we start with examples of discrete C-TMLE for variable selection, using greedy forward searching, and scalable discrete C-TMLE with pre-ordering option.

```
library(ctmle)
#> Loading required package: SuperLearner
#> Loading required package: nnls
#> Super Learner
#> Version: 2.0-22
#> Package created on 2017-07-18
#> Loading required package: tmle
#> Welcome to the tmle package, version 1.2.0-5
#>
#> Use tmleNews() to see details on changes and bug fixes
#> Loading required package: glmnet
#> Loading required package: Matrix
#> Loading required package: foreach
#> Loaded glmnet 2.0-10
library(dplyr)
#>
#> Attaching package: 'dplyr'
#> The following objects are masked from 'package:stats':
#>
#>   filter, lag
#> The following objects are masked from 'package:base':
#>
#>   intersect, setdiff, setequal, union
set.seed(123)

N <- 1000
p = 5
Wmat <- matrix(rnorm(N * p), ncol = p)
beta1 <- 4+2*Wmat[,1]+2*Wmat[,2]+2*Wmat[,5]
beta0 <- 2+2*Wmat[,1]+2*Wmat[,2]+2*Wmat[,5]
tau <- 2
gcoef <- matrix(c(-1,-1,rep(-(3/((p)-2)), (p)-2)), ncol=1)
W <- as.matrix(Wmat)

g <- 1/(1+exp(W%*%gcoef /3))
A <- rbinom(N, 1, prob = g)

epsilon <- rnorm(N, 0, 1)
Y <- beta0 + tau * A + epsilon

# With initial estimate of Q
```



```

Q <- cbind(rep(mean(Y[A == 0]), N), rep(mean(Y[A == 1]), N))

time_greedy <- system.time(
  ctmle_discrete_fit1 <- ctmleDiscrete(Y = Y, A = A, W = data.frame(Wmat), Q = Q,
    preOrder = FALSE, detailed = TRUE)
)
ctmle_discrete_fit2 <- ctmleDiscrete(Y = Y, A = A, W = data.frame(Wmat),
  preOrder = FALSE, detailed = TRUE)

time_preorder <- system.time(
  ctmle_discrete_fit3 <- ctmleDiscrete(Y = Y, A = A, W = data.frame(Wmat), Q = Q,
    preOrder = TRUE,
    order = rev(1:p), detailed = TRUE)
)

```

Scalable (discrete) C-TMLE takes much less computation time:

```

time_greedy
#>   user  system elapsed
#>  1.589   0.045   1.646
time_preorder
#>   user  system elapsed
#>  0.994   0.012   1.008

```

Show the brief results from greedy CTMLE:

```

ctmle_discrete_fit1
#> C-TMLE result:
#> parameter estimate: 1.99472
#> estimated variance: 0.00838
#> p-value: <2e-16
#> 95% conf interval: (1.81533, 2.1741)

```

Summary function offers detail information of which variable is selected.

```

summary(ctmle_discrete_fit1)
#>
#> Number of candidate TMLE estimators created: 6
#> A candidate TMLE estimator was created at each move, as each new term
#> was incorporated into the model for g.
#> -----
#>      term added cleverCovar estimate cv-RSS cv-varIC cv-penRSS
#> cand 1 (intercept)      1    4.22   19.9  0.0788   14045
#> cand 2           X2      1    3.22   19.6  0.0851   13818
#> cand 3           X5      1    2.61   19.1  0.0870   13485
#> cand 4           X1      1    2.00   18.3  0.0955   12945
#> cand 5           X4      2    1.99   18.3  0.0950   12937
#> cand 6           X3      3    2.01   18.3  0.1008   12941
#> -----
#> Selected TMLE estimator is candidate 5
#>
#> Each TMLE candidate was created by fluctuating the initial fit,  $Q_0(A,W)=E[Y|A,W]$ , obtained in stage 1.
#>
#> cand 1:  $Q_1(A,W) = Q_0(A,W) + \text{epsilon1a} * h_{1a}$ 
#>      h1a is based on an intercept-only model for treatment mechanism  $g(A,W)$ 
#>
#> cand 2:  $Q_2(A,W) = Q_0(A,W) + \text{epsilon1b} * h_{1b}$ 
#>      h1b is based on a treatment mechanism model containing covariates X2
#>
#> cand 3:  $Q_3(A,W) = Q_0(A,W) + \text{epsilon1c} * h_{1c}$ 
#>      h1c is based on a treatment mechanism model containing covariates X2, X5
#>
#> cand 4:  $Q_4(A,W) = Q_0(A,W) + \text{epsilon1d} * h_{1d}$ 
#>      h1d is based on a treatment mechanism model containing covariates X2, X5, X1
#>
#> cand 5:  $Q_5(A,W) = Q_0(A,W) + \text{epsilon1d} * h_{1d} + \text{epsilon2} * h_2$  =  $Q_4(A,W) + \text{epsilon2} * h_2$ ,
#>      h2 is based on a treatment mechanism model containing covariates X2, X5, X1, X4
#>

```

```
#>      cand 6: Q6(A,W) = Q0(A,W) + epsilon1d * h1d + epsilon2 * h2 + epsilon3 * h3      = Q5(A,W) + epsilon3 * l
#>      h3 is based on a treatment mechanism model containing covariates X2, X5, X1, X4, X3
#>
#> -----
#> C-TMLE result:
#>   parameter estimate:  1.99472
#>   estimated variance:  0.00838
#>      p-value:  <2e-16
#>   95% conf interval: (1.81533, 2.1741)
```

LASSO-C-TMLE for model selection of LASSO

In this section, we introduce the LASSO-C-TMLE algorithm for model selection of LASSO in the estimation of the propensity score. We implemented three variations of the LASSO-C-TMLE algorithm. For simplicity, we call them C-TMLE1-3. See technical details in the corresponding references.

```
# Generate high-dimensional data
set.seed(123)

N <- 1000
p = 100
Wmat <- matrix(rnorm(N * p), ncol = p)
beta1 <- 4 + 2 * Wmat[,1] + 2 * Wmat[,2] + 2 * Wmat[,5] + 2 * Wmat[,6] + 2 * Wmat[,8]
beta0 <- 2 + 2 * Wmat[,1] + 2 * Wmat[,2] + 2 * Wmat[,5] + 2 * Wmat[,6] + 2 * Wmat[,8]
tau <- 2
gcoef <- matrix(c(-1,-1,rep(-(3/((p)-2)), (p)-2)), ncol=1)
W <- as.matrix(Wmat)

g <- 1/(1+exp(W%*%gcoef /3))
A <- rbinom(N, 1, prob = g)

epsilon <- rnorm(N, 0, 1)
Y <- beta0 + tau * A + epsilon

# With initial estimate of Q
Q <- cbind(rep(mean(Y[A == 0]), N), rep(mean(Y[A == 1]), N))

glmnet_fit <- cv.glmnet(y = A, x = W, family = 'binomial', nlambda = 20)
```

We start build a sequence of lambdas from the lambda selected by cross-validation, as the model selected by cv.glmnet would over-smooth w.r.t. the target parameter.

```
lambdas <- glmnet_fit$lambda[(which(glmnet_fit$lambda==glmnet_fit$lambda.min)):length(glmnet_fit$lambda)]
```

We fit C-TMLE1 algorithm by feed the algorithm with a vector of lambda, in decreasing order:

```
time_ctmlelasso1 <- system.time(
  ctmle_fit1 <- ctmleGlmnet(Y = Y, A = A,
    W = data.frame(W = W),
    Q = Q, lambdas = lambdas, ctmletype=1,
    family="gaussian", gbound=0.025, V=5)
)
```

We fit C-TMLE2 algorithm:

```
time_ctmlelasso2 <- system.time(
  ctmle_fit2 <- ctmleGlmnet(Y = Y, A = A,
    W = data.frame(W = W),
    Q = Q, lambdas = lambdas, ctmletype=2,
    family="gaussian", gbound=0.025, V=5)
)
```

For C-TMLE3, we need two gn estimators, one with lambda selected by cross-validation, and the other with lambda slightly different from the selected lambda:

```
gcv <- predict.cv.glmnet(glmnet_fit, newx=W, s="lambda.min",type="response")
gcv <- bound(gcv,c(0.025,0.975))

s_prev <- glmnet_fit$lambda[(which(glmnet_fit$lambda == glmnet_fit$lambda.min))] * (1+5e-2)
gcvPrev <- predict.cv.glmnet(glmnet_fit,newx = W,s = s_prev,type="response")
gcvPrev <- bound(gcvPrev,c(0.025,0.975))

time_ctmlelasso3 <- system.time(
  ctmle_fit3 <- ctmleGlmnet(Y = Y, A = A, W = W, Q = Q,
    ctmletype=3, g1W = gcv, g1WPrev = gcvPrev,
    family="gaussian",
    gbound=0.025, V = 5)
)
```

Let's compare the running time for each LASSO-C-TMLE

```
time_ctmlelasso1
#>   user  system elapsed
#> 15.005   0.104  15.266
time_ctmlelasso2
#>   user  system elapsed
#> 18.351   0.083  18.528
time_ctmlelasso3
#>   user  system elapsed
#>  0.005   0.000   0.006
```

Finally, we compare three C-TMLE estimates:

```
ctmle_fit1
#> C-TMLE result:
#> parameter estimate: 2.20368
#> estimated variance: 0.09796
#> p-value: 1.9124e-12
#> 95% conf interval: (1.59022, 2.81714)
ctmle_fit2
#> C-TMLE result:
#> parameter estimate: 2.16669
#> estimated variance: 0.05327
#> p-value: <2e-16
#> 95% conf interval: (1.71429, 2.61908)
ctmle_fit3
#> C-TMLE result:
#> parameter estimate: 2.02388
#> estimated variance: 0.04972
#> p-value: <2e-16
#> 95% conf interval: (1.58684, 2.46093)
```

Show which regularization parameter (lambda) is selected by C-TMLE1:

```
lambdas[ctmle_fit1$best_k]
#> [1] 0.004409285
```

In comparison, we show which regularization parameter (lambda) is selected by cv.glmnet:

```
glmnet_fit$lambda.min
#> [1] 0.03065303
```

Advanced topic: the general template of C-TMLE

In this section, we briefly introduce the general template of C-TMLE. In this function, the gn candidates could be a user-specified matrix, each column stand for the estimated PS for each unit. The estimators should be ordered by their empirical fit.

As C-TMLE requires cross-validation, it needs two gn estimate: one from cross-validated prediction, one from a vanilla prediction. For example, consider 5-folds cross-validation, where argument `folds` is the list of indices for each folds, then the (i,j) -th element in input `gn_candidates_cv` should be the predicted value of i -th unit, predicted by j -th unit, trained by other 4 folds where all of them do not contain i -th unit. `gn_candidates` should be just the predicted PS for each estimator trained on the whole data.

We could easily use `SuperLearner` package and `build_gn_seq` function to easily achieve this:

```
lasso_fit <- cv.glmnet(x = as.matrix(W), y = A, alpha = 1, nlambda = 100, nfolds = 10)
lasso_lambdas <- lasso_fit$lambda[lasso_fit$lambda <= lasso_fit$lambda.min][1:5]

# Build SL template for glmnet
SL.glmnet_new <- function(Y, X, newX, family, obsWeights, id, alpha = 1,
                          nlambda = 100, lambda = 0,...){
  # browser()
  if (!is.matrix(X)) {
    X <- model.matrix(~-1 + ., X)
    newX <- model.matrix(~-1 + ., newX)
  }
  fit <- glmnet::glmnet(x = X, y = Y,
                        lambda = lambda,
                        family = family$family, alpha = alpha)
  pred <- predict(fit, newx = newX, type = "response")
  fit <- list(object = fit)
  class(fit) <- "SL.glmnet"
  out <- list(pred = pred, fit = fit)
  return(out)
}

# Use a sequence of estimator to build gn sequence:
SL.cv1lasso <- function (... , alpha = 1, lambda = lasso_lambdas[1]){
  SL.glmnet_new(... , alpha = alpha, lambda = lambda)
}

SL.cv2lasso <- function (... , alpha = 1, lambda = lasso_lambdas[2]){
  SL.glmnet_new(... , alpha = alpha, lambda = lambda)
}

SL.cv3lasso <- function (... , alpha = 1, lambda = lasso_lambdas[3]){
  SL.glmnet_new(... , alpha = alpha, lambda = lambda)
}

SL.cv4lasso <- function (... , alpha = 1, lambda = lasso_lambdas[4]){
  SL.glmnet_new(... , alpha = alpha, lambda = lambda)
}

SL.library = c('SL.cv1lasso', 'SL.cv2lasso', 'SL.cv3lasso', 'SL.cv4lasso', 'SL.glm')
```

Construct the object `folds`, which is a list of indices for each fold

```
V = 5
folds <- by(sample(1:N,N), rep(1:V, length=N), list)
```

Use `folds` and `SuperLearner` template to compute `gn_candidates` and `gn_candidates_cv`

```
gn_seq <- build_gn_seq(A = A, W = W, SL.library = SL.library, folds = folds)
#> Number of covariates in A11 is: 100
#> CV SL.cv1lasso_A11
#> CV SL.cv2lasso_A11
#> CV SL.cv3lasso_A11
#> CV SL.cv4lasso_A11
#> CV SL.glm_A11
#> Number of covariates in A11 is: 100
#> CV SL.cv1lasso_A11
#> CV SL.cv2lasso_A11
#> CV SL.cv3lasso_A11
#> CV SL.cv4lasso_A11
#> CV SL.glm_A11
#> Number of covariates in A11 is: 100
#> CV SL.cv1lasso_A11
```

```
#> CV SL.cv2lasso_All
#> CV SL.cv3lasso_All
#> CV SL.cv4lasso_All
#> CV SL.glm_All
#> Number of covariates in All is: 100
#> CV SL.cv1lasso_All
#> CV SL.cv2lasso_All
#> CV SL.cv3lasso_All
#> CV SL.cv4lasso_All
#> CV SL.glm_All
#> Number of covariates in All is: 100
#> CV SL.cv1lasso_All
#> CV SL.cv2lasso_All
#> CV SL.cv3lasso_All
#> CV SL.cv4lasso_All
#> CV SL.glm_All
#> Non-Negative least squares convergence: TRUE
#> full SL.cv1lasso_All
#> full SL.cv2lasso_All
#> full SL.cv3lasso_All
#> full SL.cv4lasso_All
#> full SL.glm_All
```

Lets look at the output of `build_gn_seq`

```
gn_seq$gn_candidates %>% dim
#> [1] 1000    5
gn_seq$gn_candidates_cv %>% dim
#> [1] 1000    5
gn_seq$ folds %>% length
#> [1] 5
```

Then we could use `ctmleGeneral` algorithm. As input estimator is already trained, it is much faster than previous C-TMLE algorithms.

Note: we recommend use the same `folds` as `build_gn_seq` for `ctmleGeneral`, to make cross-validation objective.

```
ctmle_general_fit1 <- ctmleGeneral(Y = Y, A = A, W = W, Q = Q,
                                  ctmle_type = 1,
                                  gn_candidates = gn_seq$gn_candidates,
                                  gn_candidates_cv = gn_seq$gn_candidates_cv,
                                  folds = folds, V = 5)

ctmle_general_fit1
#> C-TMLE result:
#> parameter estimate: 2.19494
#> estimated variance: 0.08348
#> p-value: 3.0302e-14
#> 95% conf interval: (1.62865, 2.76122)
```

Citation

If you used `ctmle` package in your research, please cite:

Ju, Cheng; Susan, Gruber; van der Laan, Mark J.; ctmle: Collaborative Targeted Maximum Likelihood Estimation. R package version 0.1.1, <https://CRAN.R-project.org/package=ctmle>.

```
@Manual{,
  title = {ctmle: Collaborative Targeted Maximum Likelihood Estimation},
  author = {Cheng Ju and Susan Gruber and Mark van der Laan},
  year = {2017},
  note = {R package version 0.1.1},
  url = {https://CRAN.R-project.org/package=ctmle},
}
```

References (by inverse chronological order)

C-TMLE for Adaptive Propensity Score Truncation

Ju, Cheng, Schwab, Joshua, & van der Laan, Mark J. (2017). On Adaptive Propensity Score Truncation in Causal Inference. arXiv preprint arXiv:1707.05861 (2017).

LASSO-C-TMLE

Ju, Cheng; Wyss, Richard; Franklin, Jessica M.; Schneeweiss, Sebastian; Häggström, Jenny; van der Laan, Mark J.. "Collaborative-controlled LASSO for Constructing Propensity Score-based Estimators in High-Dimensional Data", arXiv preprint arXiv: 1706.10029 (2017). (To appear in Statistical Methods in Medical Research)

Scalable Discrete C-TMLE with Pre-ordering

Ju, Cheng; Gruber, Susan; Lendle, Samuel D.; Chambaz, Antoine; Franklin, Jessica M.; Wyss, Richard; Schneeweiss, Sebastian; and van der Laan, Mark J.. "Scalable Collaborative Targeted Learning for High-dimensional Data", Statistical Methods in Medical Research (2017), <https://doi.org/10.1177/0962280217729845>.

Discrete C-TMLE with Greedy Search

Susan, Gruber, and van der Laan, Mark J.. "An Application of Collaborative Targeted Maximum Likelihood Estimation in Causal Inference and Genomics." The International Journal of Biostatistics 6.1 (2010): 1-31.

General Template of C-TMLE

van der Laan, Mark J., and Susan Gruber. "Collaborative double robust targeted maximum likelihood estimation." The international journal

Releases

No releases published

Packages

No packages published

Contributors 2

 jucheng1992 Cheng

 MengxinJi Mengxin

Languages

● R 100.0%